

Unsupervised Steering Vectors from Learned Classifiers: A Summary

Keira Wiechecki
WAKARU · KAW504@NYU.EDU

Lamb Wakefield
FLOCK CAPITAL · LAMB@FLOCK.HOLDINGS

Jade Zaslavsky
WAKARU · JADEOFNAMELESS@GMAIL

vmfunc
FLOCK CAPITAL · CELESTE@LINUX.COM

October 12, 2026

Checking a model for a hidden goal, or for a concept no one thought to name, means finding the model's internal variables without anyone labeling them first. This summary describes a method built for that and says plainly where it stands: it describes a network's internal state compactly enough for the network to run on, and editing that description moves the network's output specifically, but past the first round the variables it finds are not the same from one training run to the next, and until they are, they cannot be named or relied on.

What the method does

The method, which we call an *Ontologizer*, rewrites a neural network's internal representation at one point as a short list of discrete choices, and rebuilds the representation from them. It is made of small classifiers. Each classifier sorts every input into one of k categories, and each category stores one vector; the input is rebuilt as the sum of the vectors its categories select. In the models described here there are 5 rounds of 32 classifiers with 32 categories each, so an input is described by 160 choices — 800 bits — plus one number per round giving its overall size. Nothing is labeled: classifiers and vectors are learned only by asking the rebuilt representation to match the original.

Because a classifier's categories are mutually exclusive, changing one classifier's choice for an input — from category j to category j' — swaps one stored vector for another. That swap is the steering move the method is built around: it moves the representation from j to j' along one axis the method found, and the result

is always another valid description, so the edited representation stays within the range the network normally sees.

We study two representations. The main one is GPT-2 small's internal state after its first eight blocks [1], on web text: there the rebuilt state can be put back into the running model, so we can measure how much of the model's behavior survives. The second is SONAR sentence embeddings [2]: a fixed-length vector per text, here the opening of a web document (SONAR is built for sentences), that can be decoded back into a short paraphrase, which makes a steered embedding readable.

What it is for

Watching what a model does cannot settle what it is trying to do: a model can behave differently when it believes it is being trained [3]. Settling it means reading the model's internal state, including concepts no one has named.

To steer a model is to change its internal state so that it behaves differently in a chosen way. A standard method is a *steering vector*: average the representations of examples with a property, subtract the average of examples without it, and add the difference to new inputs [4]; a single contrasting pair of prompts can be enough [5]. This often works, though not for every property, and someone who already suspects the property exists must supply contrasting examples of it. Supervised steering, and the linear probe that detects a property the same way, can therefore confirm a concept someone suspects but cannot find one nobody does.

Sparse autoencoders (SAEs) find features without labels: each input is described by a few active features out of thousands [6, 7]. But each feature is an isolated direction that is either on or off, with no stated alternatives. A feature can stay off on inputs that have it, because a more specific feature has taken over its direction [8], so its silence says little; features split as the dictionary grows [9]; and adding a feature direction can push the representation anywhere. On a recent steering benchmark, prompting, fine-tuning and the averaged difference each beat them [10].

The methods that win are contrasts between alternatives: a property against its absence, one value of a *variable* — a property with several possible values, such as a text's topic — against another. The goal is steering vectors that need no labels but keep that shape: each one a move between named alternatives, found together with the contrast it moves along. Steering then doubles as a test of the method: a swap that moves the model's behavior the way the categories predict is evidence that the variable matters to the model and not only to the rebuilt representation.

What it assumes, and why that is acceptable

1. **A representation is mostly a sum of a few categorical choices, plus a remainder that later rounds describe.** Much of language is categorical — which token, which syntactic role, which topic — and describing a remainder

in successive rounds is how residual vector quantization works in compression [11]. On GPT-2, 800 bits per token rebuild the held-out state well enough that, put back into the model, it recovers 97.5% of the next-token loss that replacing the state with its average would cost.

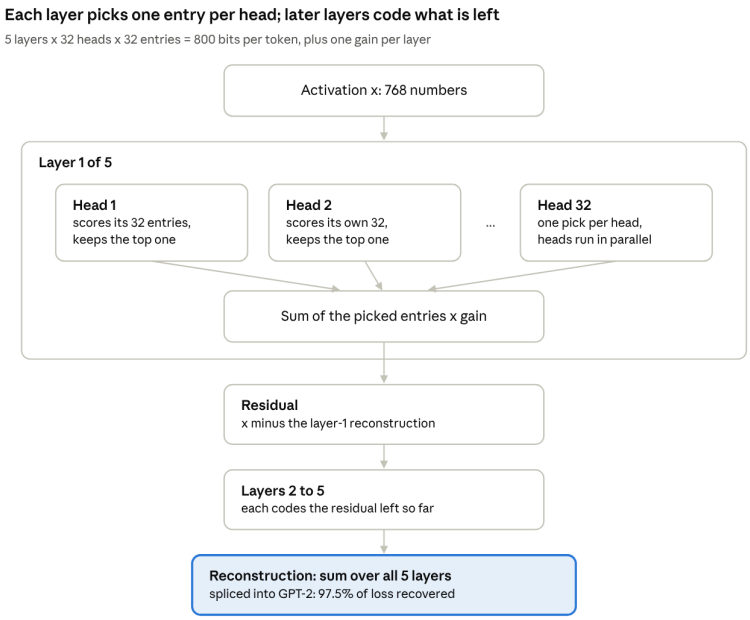
2. **The classifiers act as roughly independent factors.** Independence is what lets a few small choices cover many combinations. On GPT-2 the information shared between two classifiers' choices is 0.27 bits in the first round and 0.02 bits by the third, against about 5 bits in each choice.

3. **The network tolerates running on the description.** If it did not, editing the description would mean nothing. Beyond the 97.5% recovery, swapping one classifier's choice changes GPT-2's next-token distribution by only 0.01–0.06 nats.

4. **The categories can be named.** Partly true. On GPT-2, a judge model shown a description of a first-round category picks out its most typical members from distractors 85–100% of the time — but only 55–69% for randomly chosen members, and at chance in the fourth and fifth rounds. On sentence embeddings, the strongest classifier sorts by topic and genre (crime news, legislation, encyclopedic description, and so on).

5. **The categories are a property of the data, not of the training run.** This has *not* held. Two runs from different random starts reconstruct equally well, but on both representations only the first round's classifiers recur; later rounds differ from run to run.

How it works, intuitively



Think of describing a token by answering a fixed list of multiple-choice questions, where the questions and the answers are learned. In the first round, 32 classifiers each pick their best category in parallel, and the picked vectors are summed and scaled to the size of what they explain. The next round sees only what the first got wrong, and describes what remains; five rounds in all. Training uses hard choices in the forward pass and soft ones when computing gradients (a straight-through estimator). The design is a residual quantizer in which each round adds up vectors picked from several sets [11, 12] (product quantization [13] is the case where each set covers its own coordinates) — the compression literature's closest relative — used not for compression but so that every choice can be read and swapped. The same kind of quantizer has been used to turn SONAR embeddings into discrete codes for a language model that writes in them [14].

How it compares with existing methods

Against supervised steering — the comparison that matters. Three tests so far, none of them yet the decisive one.

- *Swaps are specific (GPT-2).* Swapping a token's category from j to j' moves the model's output toward j''s typical output: among 31 alternatives, j' ranks 1.4–4 on average, and still 2–6 at four times the step, where a random step of the same size ranks 10–16. That is specificity against an internal control, not success at a named concept.

- *Language (sentence embeddings).* Forcing one of the method's categories moved decoded text toward a target language 0.64 times as far as the supervised difference-of-means vector did, and 4 times as far as a linear-probe direction. All methods, supervised included, damaged unrelated content about as much as a random step of the same size.

- *Reaching its own categories (sentence embeddings).* For each category we fitted the two standard supervised vectors — difference of means and a linear probe — using the method's own assignments as labels (a generous baseline: it already knows the answer), and gave every direction the same step. The soft version's own direction reached its category on 75% of inputs, against 46–47% for the supervised vectors; the hard multi-round version's reached only 37%, against 91–94%, so its categories are reachable but its own direction does not point at them.

The decisive test — steering toward externally defined concepts, with supervised vectors fitted to those concepts' labels, SAE features and the method's swaps scored by the same judge for the concept, for staying on task and for fluency, each method at its best tuned strength, as in AxBench [10] — has not been run.

Against SAEs on steering. On sentence embeddings, at each model's own step size, a feature of a standard SAE (32 active per input) takes effect on 46–50% of inputs and the multi-round method on 8–19%, both disturbing other features or classifiers about as much as a random step; a one-round version of the method takes effect on 41% while disturbing 1.4% of its other classifiers, against 18% for a random step. On GPT-2 only removal has been compared: setting one classifier to its average choice everywhere costs 0.07 nats, removing one SAE feature where active 0.055.

Against SAEs on reconstruction (a precondition, not the goal). Steering through a description only means something if the model runs on it.

GPT-2 small, after 8 blocks, held out	per-token code	unexplained variance	loss recovered
Ontologizer, 32 categories	800 bits + 5 numbers	0.186	97.5%
Ontologizer, 64 categories	960 bits + 5 numbers	0.164	97.9%
SAE, 32 active features	32 indices + 32 numbers	0.189	96.2%
SAE, 128 active features	128 indices + 128 numbers	0.116	98.6%

At nearly the same unexplained variance the method recovers more of the model's loss than the matching SAE, so its errors fall where the model cares less. On sentence embeddings, 1900 bits of choices and the per-round sizes leave 15% of the variance unexplained, and 16% with the sizes fixed. A standard SAE whose description is held to the same 1900 bits, about 1200 saying which 160 features are active and the rest for their 160 numbers, leaves at least 24% however those numbers are rounded. But an SAE trained with a weaker sparsity penalty, its

numbers rounded coarsely and each feature's activity coded by how often it fires, leaves 13% at 1900 bits: its features fire unevenly, which makes the list of active ones cheap to send, while the method's categories are used evenly and leave nothing to save.

What we can claim

On both representations the method describes the network's state with a small set of discrete choices about as accurately as a sparse autoencoder that also sends continuous numbers (more accurately than the standard kind at the same number of bits, though not than every kind), and on GPT-2 the model runs on that description with little loss. Swapping one choice moves the model's output specifically toward the new category. Some categories can be named: on GPT-2 a first-round category's most typical members, and — more promisingly — combinations of two first-round categories, which a judge tells apart from the rest of the category 77–96% of the time, against 48–50% for random splits of the same size; on sentence embeddings, a topic-and-genre classifier that recurs in every run. Whether the categories are in general easier to interpret than SAE features, and whether swapping them steers better than supervised vectors toward concepts a user cares about, is not established.

What does not work yet

- **The classifiers past the first round are not reproducible.** This is the main open problem, not a detail. Two runs reconstruct equally well but agree on almost none of these classifiers, on both representations, so reconstruction quality cannot reveal the failure. A classifier that differs on every run cannot be named, an audit could not rely on it, and a goal defined in terms of it would not keep pointing at the same thing once the model is retrained.
- **Categories are named from their extremes.** Descriptions fit a category's most typical members, not its ordinary ones, the failure automated description shows when it sees only the strongest examples [15], and the deeper rounds cannot yet be named at all — though they are causal and steer specifically.
- **Steering through the input is not targeted.** On sentence embeddings, pushing an input toward one category changes other classifiers' choices about as much as a random push of the same size. Swaps made on the description itself do not have this problem.

Next

Reproducibility comes first: making the classifiers reproducible, or identifying the ones that are. One first-round classifier on sentence embeddings already recurs in every run, and what lets it recur when the others do not is the first question. Alongside it comes testing whether combinations of categories — rather than single ones — are the right unit for naming the deeper rounds.

The test the method then needs is AxBench-style steering toward externally defined concepts [10]: difference-of-means vectors from labeled examples, SAE features and the method's swaps, each at its best tuned strength, scored by a judge for the concept, for staying on task and for fluency, plus detection against a linear probe. That benchmark targets Gemma 2, which would mean moving off GPT-2.

References

[1] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. OpenAI, 2019.

[2] Paul-Ambroise Duquenne, Holger Schwenk, and Benoît Sagot. *SONAR: Sentence-Level Multimodal and Language-Agnostic Representations*. 2023. arXiv: 2308.11466 [cs.CL]. URL: https://arxiv.org/abs/2308.11466.

[3] Ryan Greenblatt et al. *Alignment Faking in Large Language Models*. 2024. arXiv: 2412.14093 [cs.AI]. URL: https://arxiv.org/abs/2412.14093.

[4] Nina Rimsky et al. "Steering Llama 2 via Contrastive Activation Addition". In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Bangkok, Thailand: Association for Computational Linguistics, 2024, pp. 15504–15522. DOI: 10.18653/v1/2024.acl-long.828. URL: https://aclanthology.org/2024.acl-long.828/.

[5] Alexander Matt Turner et al. *Steering Language Models With Activation Engineering*. 2024. arXiv: 2308.10248 [cs.CL]. URL: https://arxiv.org/abs/2308.10248.

[6] Trenton Bricken et al. "Towards Monosemanticity: Decomposing Language Models With Dictionary Learning". In: *Transformer Circuits Thread* (2023). URL: https://transformer-circuits.pub/2023/monosemantic-features/index.html.

[7] Leo Gao et al. *Scaling and evaluating sparse autoencoders*. 2024. arXiv: 2406.04093 [cs.LG]. URL: https://arxiv.org/abs/2406.04093.

[8] David Chanin et al. "A is for Absorption: Studying Feature Splitting and Absorption in Sparse Autoencoders". In: *Advances in Neural Information Processing Systems*. Vol. 38. 2025, pp. 91489–91526. arXiv: 2409.14507 [cs.CL]. URL: https://arxiv.org/abs/2409.14507.

[9] Bart Bussmann et al. “Learning Multi-Level Features with Matryoshka Sparse Autoencoders”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 6077–6101. arXiv: 2503.17547 [cs.LG]. URL: https://arxiv.org/abs/2503.17547.

[10] Zhengxuan Wu et al. “AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025, pp. 67035–67080. arXiv: 2501.17148 [cs.CL]. URL: https://arxiv.org/abs/2501.17148.

[11] Neil Zeghidour et al. *SoundStream: An End-to-End Neural Audio Codec*. 2021. arXiv: 2107.03312 [cs.SD]. URL: https://arxiv.org/abs/2107.03312.

[12] Artem Babenko and Victor Lempitsky. “Additive Quantization for Extreme Vector Compression”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2014, pp. 931–938. URL: https://openaccess.thecvf.com/content_cvpr_2014/html/Babenko_Additive_Quantization_for_2014_CVPR_paper.html.

[13] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. “Product Quantization for Nearest Neighbor Search”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33.1 (2011), pp. 117–128.

[14] LCM team et al. *Large Concept Models: Language Modeling in a Sentence Representation Space*. 2024. arXiv: 2412.08821 [cs.CL]. URL: https://arxiv.org/abs/2412.08821.

[15] Gonçalo Paulo et al. “Automatically Interpreting Millions of Features in Large Language Models”. In: *Proceedings of the 42nd International Conference on Machine Learning*. Vol. 267. Proceedings of Machine Learning Research. PMLR, 2025. arXiv: 2410.13928 [cs.LG]. URL: https://arxiv.org/abs/2410.13928.